CS3307 Final Documentation

Members:

Ming Chun Chan
251306826
mchan624@uwo.ca

Andrei Ciceu
251355626
aciceu@uwo.ca

Architecture Overview

Presentation Layer

Class	Methods
MainWindow	ui: Ui::MainWindow*manager: flashcardManager, deckbuttons: QVector<QPushButton*>, currentPage: int = 0, decksPerPage: int = 9, currentTagFilter: QString = ”” MainWindow(), ~MainWindow(), onCreateDeckClicked(), onDeckButtonClicked(), onNextPageClicked(), onPrevPageClicked(), onDeleteDeckClicked(), onRenameDeckClicked(), setupDeckButtons(), refreshDeckButtons(), totalPages(), getFilteredDeckNames(), onImportDeckClicked(), onExportDeckClicked(), onExportAllDecksClicked()
DeckWindow	ui: Ui::DeckWindow*, m_deck: deck* = nullptr, m_current: int = -1, studywindow: studywindow, m_model: QStringListModel* = nullptr DeckWindow(), ~DeckWindow(), onAddButtonClicked(), onUpdateButtonClicked(), onDeleteButtonClicked(), onStudyButtonClicked(), onPrevButtonClicked(), onNextButtonClicked(), onRowChanged(), onClosedButtonClicked(), rebuildList(), loadFieldsFromCurrent(), setCurrentIndex()
StudyWindow	ui: Ui::studywindow*, *currentdeck: deck, currentIndex: int

	studywindow(), ~studywindow(), onCheckAnswerClicked(), onNextCardClicked(), updateCardDisplay(), onReturnClicked()

Business Logic Layer

Class	Attributes/Methods
FlashcardManager	decks: QMap<QString, deck> flashcardManager(), addDeck(), getDeck(), removeDeck(), getDeckNames(), saveToDisk(), loadFromDisk(), exportDeckToFile(), exportAllDecksToFile(), importDeckFromFile(), storageFilePath: QString
FlashcardFactory	create()
StatsTracker	settings: QSettings instance(),trackDeckCreated(), trackCardCreated(), trackReview(), trackCorrectAnswer(), trackIncorrectAnswer(), getTotalDecks(), getTotalCards(), getTotalReviews(), getTotalCorrect(), getTotalIncorrect(), resetStats()

Data Layer

Class	Attributes/Methods
Flashcard	question : QString, answer : QString flashcard(), getQuestion(), getAnswer(), setQuestion(), setAnswer()
Deck	name: QString, tag: QString, cards: QVector<Flashcard> deck(), addCard(), getCard(), getSize(), getName(), setName(), updateCard(), removeCard(), setTag(), getTag()

Design Patterns

Creational Design Patterns:

1) Singleton (Creational) — flashcardManager, StatsTracker

What it is: Singleton ensures only one instance of a class exists and provides a global access point to it.

Why we chose it:
The application needs exactly one "source of truth" for:

- all decks/flashcards (data model), and
- application-wide statistics (reviews, correct/incorrect, totals).

If multiple instances existed, different windows could accidentally read/write different copies of the data or stats, causing inconsistent behavior.

How it improved the system:

- Ensures **consistent state** across all windows (MainWindow, DeckWindow, StudyWindow).
- Prevents duplication/desynchronization of decks and stats.
- Simplifies access (e.g., flashcardManager::instance() and StatsTracker::instance()), so UI code doesn't need to pass these objects through many constructors.

2) Factory Method (Creational) — FlashcardFactory

What it is: Factory Method centralizes object creation so calling code does not directly construct concrete types.

Why we chose it:
Deck creation/editing happens in UI code (DeckWindow). If UI code directly constructs

flashcards everywhere, any future change to flashcard types (e.g., multiple-choice, image-based, fill-in-the-blank) would require editing multiple UI locations. A factory isolates creation logic into one place.

How it improved the system:

- Decouples "**how a flashcard is created**" from "**where it's used**".
- Makes the code more maintainable: changes to construction logic happen in one class instead of many UI files.
- Enables extensibility: the UI can request a flashcard type and the factory decides which concrete implementation to create.

Structural/Behavioral Pattern:

3) Qt Model–View (Behavioral / Architectural) — QStringListModel + QListView

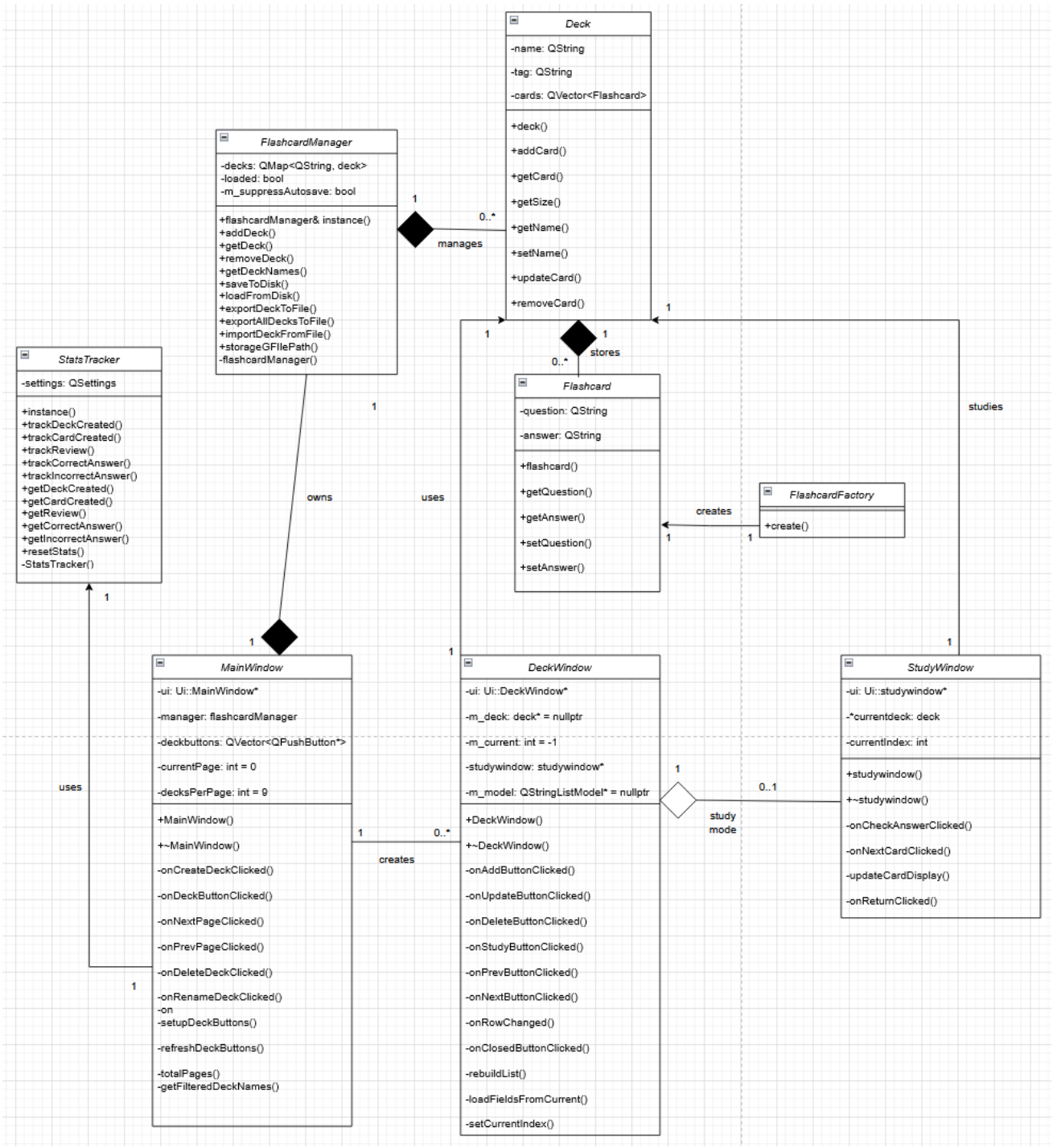
What it is: Qt's Model–View architecture separates data (model) from presentation (view). The view displays data stored in the model and reacts to model changes.

Why we chose it:
The deck contents and deck list need to be displayed and updated frequently. Using Model–View avoids manually repainting UI elements and reduces coupling between UI and data storage.

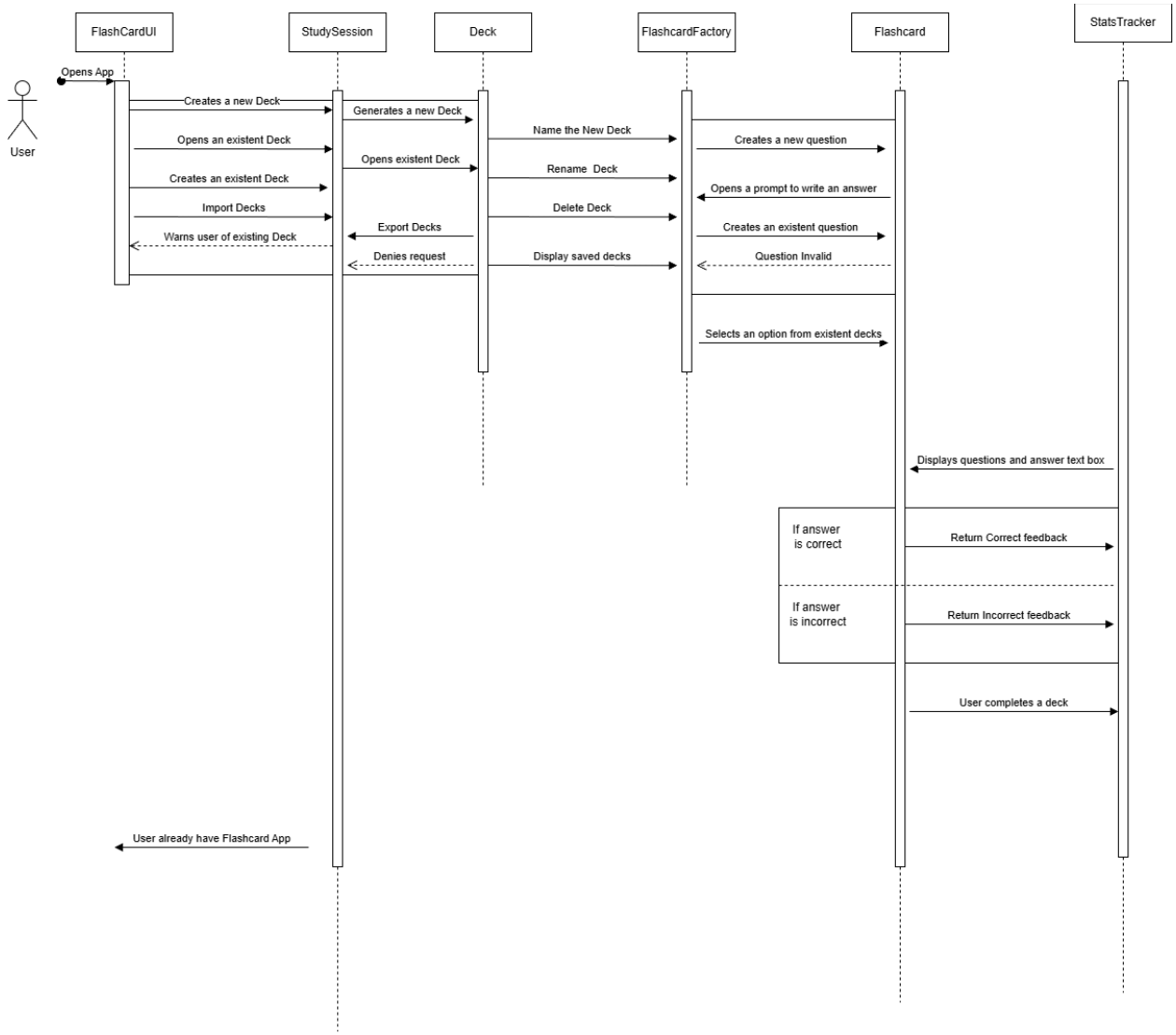
How it improved the system:

- Separation of concerns: the **model stores data**, the **view displays it**.
- Easier updates: refreshing the model updates the UI consistently.
- Cleaner UI logic: less manual widget management and fewer error-prone updates.

Class Diagram


Deck
-name: QString
-tag: QString
-cards: QVector<Flashcard>
+deck()
+addCard()
+getCard()
+getSize()
+getName()
+setName()
+updateCard()
+removeCard()
FlashcardManager
-decks: QMap<QString, deck>
-loaded: bool
-m_suppressAutosave: bool
+flashcardManager& instance()
+addDeck()
+getDeck()
+removeDeck()
+getDeckNames()
+saveToDisk()
+loadFromDisk()
+exportDeckToFile()
+exportAllDecksToFile()
+importDeckFromFile()
+storageGFIlePath()
-flashcardManager()
manages
1
0..*
stores
1
0..*
StatsTracker
-settings: QSettings
+instance()
+trackDeckCreated()
+trackCardCreated()
+trackReview()
+trackCorrectAnswer()
+trackIncorrectAnswer()
+getDeckCreated()
+getCardCreated()
+getReview()
+getCorrectAnswer()
+getIncorrectAnswer()
+resetStats()
-StatsTracker()
owns
uses
Flashcard
-question: QString
-answer: QString
+flashcard()
+getQuestion()
+getAnswer()
+setQuestion()
+setAnswer()
FlashcardFactory
+create()
creates
studies
MainWindow
-ui: Ui::MainWindow*
-manager: flashcardManager
-deckbuttons: QVector<QPushButton*>
-currentPage: int = 0
-decksPerPage: int = 9
+MainWindow()
+~MainWindow()
-onCreateDeckClicked()
-onDeckButtonClicked()
-onNextPageClicked()
-onPrevPageClicked()
-onDeleteDeckClicked()
-onRenameDeckClicked()
-on
-setupDeckButtons()
-refreshDeckButtons()
-totalPages()
-getFilteredDeckNames()
uses
creates
DeckWindow
-ui: Ui::DeckWindow*
-m_deck: deck* = nullptr
-m_current: int = -1
-studywindow: studywindow*
-m_model: QStringListModel* = nullptr
+DeckWindow()
+~DeckWindow()
-onAddButtonClicked()
-onUpdateButtonClicked()
-onDeleteButtonClicked()
-onStudyButtonClicked()
-onPrevButtonClicked()
-onNextButtonClicked()
-onRowChanged()
-onClosedButtonClicked()
-rebuildList()
-loadFieldsFromCurrent()
-setCurrentIndex()
study
mode
0..1
StudyWindow
-ui: Ui::studywindow*
-*currentdeck: deck
-currentIndex: int
+studywindow()
+~studywindow()
-onCheckAnswerClicked()
-onNextCardClicked()
-updateCardDisplay()
-onReturnClicked()

Sequence Diagram



Use Case Diagram

Testing Report

```
19:38:06: Starting C:\Users\mcher\Desktop\CS3307\oop-assignment-3-ming-chun-andrei\tests\build\Desktop_Qt_6_10_0_MinGW_64_bit-Debug\debug\deck_tests.exe...
[==========] Running 13 tests from 4 test suites.
[----------] Global test environment set-up.
[----------] 3 tests from FlashcardManagerTest
[ RUN      ] FlashcardManagerTest.StartsEmptyAfterClear
[       OK ] FlashcardManagerTest.StartsEmptyAfterClear (2 ms)
[ RUN      ] FlashcardManagerTest.AddDeckThenGetDeck
[       OK ] FlashcardManagerTest.AddDeckThenGetDeck (2 ms)
[ RUN      ] FlashcardManagerTest.RemoveDeckWorksAndMissingDeckReturnsNull
[       OK ] FlashcardManagerTest.RemoveDeckWorksAndMissingDeckReturnsNull (0 ms)
[----------] 3 tests from FlashcardManagerTest (5 ms total)

[----------] 1 test from FlashcardManagerMockTest
[ RUN      ] FlashcardManagerMockTest.CanMockFlashcardManagerBehavior
[       OK ] FlashcardManagerMockTest.CanMockFlashcardManagerBehavior (0 ms)
[----------] 1 test from FlashcardManagerMockTest (0 ms total)

[----------] 2 tests from DeckWithMock
[ RUN      ] DeckWithMock.StreamsCardsToSinkInOrder
[       OK ] DeckWithMock.StreamsCardsToSinkInOrder (0 ms)
[ RUN      ] DeckWithMock.EmptyDeckMakesNoCalls
[       OK ] DeckWithMock.EmptyDeckMakesNoCalls (0 ms)
[----------] 2 tests from DeckWithMock (0 ms total)

[----------] 7 tests from DeckTest
[ RUN      ] DeckTest.StartsEmpty
[       OK ] DeckTest.StartsEmpty (0 ms)
[ RUN      ] DeckTest.AddCardIncreasesSizeAndPreservesOrder
[       OK ] DeckTest.AddCardIncreasesSizeAndPreservesOrder (0 ms)
[ RUN      ] DeckTest.UpdateCardValidIndexReplacesCard
[       OK ] DeckTest.UpdateCardValidIndexReplacesCard (0 ms)
[ RUN      ] DeckTest.UpdateCardInvalidIndexReturnsFalseAndDoesNotChange
[       OK ] DeckTest.UpdateCardInvalidIndexReturnsFalseAndDoesNotChange (0 ms)
[ RUN      ] DeckTest.RemoveCardValidIndexRemovesAndShifts
[       OK ] DeckTest.RemoveCardValidIndexRemovesAndShifts (0 ms)
[ RUN      ] DeckTest.RemoveCardInvalidIndexReturnsFalseAndDoesNotChange
[       OK ] DeckTest.RemoveCardInvalidIndexReturnsFalseAndDoesNotChange (0 ms)
[ RUN      ] DeckTest.SettersUpdateMetadata
[       OK ] DeckTest.SettersUpdateMetadata (0 ms)
[----------] 7 tests from DeckTest (0 ms total)

[----------] Global test environment tear-down
[==========] 13 tests from 4 test suites ran. (6 ms total)
[  PASSED  ] 13 tests.
19:38:08: The command "C:\Users\mcher\Desktop\CS3307\oop-assignment-3-ming-chun-andrei\tests\build\Desktop_Qt_6_10_0_MinGW_64_bit-Debug\debug\deck_tests.exe" finished successfully.
```

Deck (pure data structure / logic)

1) Out-of-range indices (negative and too-large)

- **Why it matters:** Methods like updateCard(index, ...), removeCard(index), and getCard(index) are common sources of crashes if bounds aren't checked.
- **How we handled it:** We included tests that call these methods with:
  - index = -1
  - index = size (just outside the valid range)
- **Expected behavior:**
  - updateCard and removeCard should return false and **not modify the deck**.
  - For getCard, we avoided invalid access in unit tests (since an unchecked access could crash the process), and instead focused on verifying that the "safe" operations reject invalid indices.

2) Empty deck behavior

- **Why it matters:** Many bugs happen when containers are empty (e.g., removing from an empty list, updating index 0 when no cards exist).
- **How we handled it:** We tested a newly created deck to confirm:
  - getSize() returns 0

    - calling “mutating” functions with invalid indices fails gracefully
- **Benefit:** This prevents segmentation faults and ensures UI code can call these operations safely when the deck is empty.

3) Data integrity after removals

- **Why it matters:** Removing elements from a QVector shifts later elements; logic errors can cause the wrong card to remain or indices to become inconsistent.
- **How we handled it:** After removing an element, we assert:
    - the size decreased by 1
    - remaining cards shifted correctly (e.g., original card 1 becomes new card 0)

4) Default fields

- **Why it matters:** Decks created via default constructor should have predictable metadata used elsewhere in the app (name/tag).
- **How we handled it:** We validated default name/tag and that the internal list starts empty.

FlashcardManager (singleton + persistence)

1) Singleton state leaking across tests

- **Why it matters:** Because flashcardManager::instance() returns the same object every time, one test can leave decks behind and break later tests.
- **How we handled it:** Each test begins by clearing state (iterating over getDeckNames() and calling removeDeck(...)) so tests remain independent and repeatable.
- **Benefit:** This avoids false failures caused by leftover decks from earlier tests.

2) Avoiding writes to real user data

- **Why it matters:** Your manager persists to QStandardPaths::AppDataLocation (real user folder). Tests must not overwrite the user’s actual decks.json.
- **How we handled it:** We enable Qt’s test mode:
    - QStandardPaths::setTestModeEnabled(true);
      before calling flashcardManager::instance().
- **Benefit:** All file I/O goes to Qt’s isolated test directory instead of the real AppData location.

3) Missing deck lookups

- **Why it matters:** UI and logic often ask for a deck that might not exist (typos, deleted deck, not loaded yet). Returning invalid pointers would be dangerous.
- **How we handled it:** We test that:
  - getDeck("Nonexistent") returns nullptr
  - removeDeck("Nonexistent") returns false
- **Benefit:** Prevents crashes and simplifies caller logic: callers can just check for nullptr / boolean.

4) Duplicate deck names

- **Why it matters:** Adding a deck with an existing name could silently overwrite, create duplicates, or cause inconsistent state depending on implementation.
- **How we handled it (discussion):** We designed tests to ensure deterministic behavior:
  - If duplicates are **disallowed**, adding a second deck with the same name should not create an extra entry (deck names remain unique).
  - If duplicates are **allowed**, then the import/export layer should rename on collision (common pattern: "Name (2)").
- **Note:** This edge case is especially important for import/export flows where user files can contain name collisions.

5) Persistence failures / corrupt JSON

- **Why it matters:** Real files can be empty, corrupted, or partially written. A robust manager should not crash when JSON parsing fails.
- **How we handled it (discussion):** Tests and error-handling design should ensure:
  - load failures return false and provide an error string (if supported)
  - the manager remains in a valid state (e.g., keeps previous decks or resets cleanly)
- **Practical result:** Even if disk state is bad, the app can still run and show an error instead of crashing.

Google Mock edge case focus (why we mocked an interface)

Because flashcardManager is a singleton with non-virtual methods, it cannot be directly mocked with gmock. Instead, we mock an interface (IFlashcardManager) that mirrors the methods we depend on. This allows edge cases like:

- dependency returning nullptr for a missing deck
- dependency reporting empty deck lists
- dependency failing removals

...and we can verify those interactions with EXPECT_CALL without depending on disk I/O or singleton state.

Reflection

The main challenge we face is patterns. We struggled a lot to implement the required patterns into our application, but we still managed to fit them in at the end. Due to time constraints, we made the trade-off to use QStringListModel and QListView to display the flashcards, instead of custom UI flashcard we are used to see in other applications where we can flip the cards. We have learned a lot about QtCreator. While we weren't able to make the actual program look overly appealing, we made sure to hit almost all of our functionality goals to make the project something that we feel is functionally complete. We are able to link up classes, create basic UI, and code effectively. Overall, we think we did a pretty good job on the Flashcard app.